

PROJETO E IMPLEMENTAÇÃO DE UM COMPILADOR- 2 PARTE

Projete e implemente um analisador sintático para a linguagem C-, definida pela gramática livre de contexto listada no fim deste documento:

1) Escolha como o parser será construído:

Parser Recursivo Descendente: converta a gramática C- em uma gramática LL(1);

Implemente um parser recursivo descendente que reconheça a gramática LL(1) resultante

Gerador de Parsers: converta a gramática C em uma gramática LR(1) ou LL(1), o que preferir.

Prepare a interface do parser gerada para a etapa de geração de árvore de sintaxe abstrata.

2) Durante o parsing crie toda a árvore de sintaxe abstrata em memória;

3) Use a solução da parte 1 do projeto como ponto inicial para a implementação

4) Adicione um esquema de recuperação de erros com sincronização.

Caso use o gerador de parsers: entenda como o gerador provê recuperação de erros. implemente no mínimo a recuperação do tipo *phrase-level*

5) Atenda a todas as especificações mínimas a seguir. Lembre-se que este trabalho faz parte de um projeto muito maior.

Documentação mínima:

A documentação a ser entregue, deve ser como um pequeno artigo objetivo, impessoal e completo que explica os detalhes do que o seu programa faz, como faz, e ainda apresenta conclusões obtidas pelo trabalho. É um documento a parte que deve conter pelo menos as seguintes estruturas [seções]:

1. Título

- a. Apresente o título do trabalho e/ou nome do compilador
- b. Nome dos integrantes do grupo.

2. Visão geral

- a. Dê uma visão geral do funcionamento do programa
- b. Descrição dos módulos e suas interdependências. Apresente uma breve descrição de cada módulo bem como um diagrama mostrando suas relações
- c. Analisador sintático:
 - i. Descreva e justifique a forma com a qual vocês implementaram o analisador sintático
 - ii. Apresente a gramática LL(1) ou LR(1) resultante
 - iii. Apresente as estruturas de dados, os métodos e os algoritmos utilizados
- d. Gerenciador de erros sintáticos:
 - i. Apresente claramente o método de gerenciamento de erros escolhido

- ii. Apresente as estruturas de dados, os métodos e os algoritmos utilizados
- e. Árvore de sintaxe abstrata:
 - i. Mostre em detalhes como a árvore de sintaxe abstrata é construída
- 3. Resultados experimentais
 - a. Listagem dos testes executados: deve conter os arquivos de entrada compilados pelo programa (arquivo de texto .txt com código C-) **com os respectivos resultados de saída**
 - b. Comente os testes executados
 - c. ATENÇÃO: Faça os testes levando em conta as diferentes construções da linguagem e as construções mais relevantes. Prove que o seu compilador é capaz de realizar as tarefas especificadas abaixo
- 4. Conclusão
 - a. Esse é o lugar em que vocês podem comunicar tudo o que vocês acham que é importante de ser dito
 - b. Descreva todas as características especiais do seu compilador nessa seção
 - c. Indique extensões e funcionalidades extras
- 5. Referências Bibliográficas
 - a. Mostre todas as fontes que vocês usaram para fazer o trabalho. Artigos, livros, links e outros materiais usados por vocês devem ser referenciados mas não precisam seguir nenhuma norma específica para citação desde que seja clara a origem do documento. Para capítulos de livros indique quais seções específicas foram usadas em cada etapa do trabalho. Todas as seções devem ser acompanhadas de ao menos três referências.

Especificações mínimas para o analisador sintático:

- Esse trabalho é realizável apenas através da leitura de livros. Não reinvente a roda. Seja objetivo e desenvolva como está na literatura, observando rigorosamente as especificações a seguir
- Utilize onde achar necessário, comentários sobre o funcionamento de partes do código
- A saída do programa deverá ser enviada para o stdout.
- As mensagens de erro devem ser enviadas para o stderr.
- A precedências dos operadores C-, da menor para a maior, e que deve ser resolvida na gramática LL(1) é:
 - a. Atribuição
 - b. Or
 - c. And
 - d. Igual e diferente
 - e. Outros relacionais
 - f. Adição e subtração
 - g. Multiplicação e divisão
 - h. Parenteses
- Ao converter a GLC C-, para LL(1) ou LR(1), estejam certos de que vocês não estão mudando a linguagem que é aceita pela gramática. Não se preocupe com

nenhuma regra sensível ao contexto. Outras restrições à linguagem serão impostas na próxima parte do projeto. De fato, há uma parte da GLC que não pode ser convertida para LL(1) ou LR(1). O problema ocorre em algumas formas de expressões envolvendo o operador de atribuição. A dificuldade está em determinar se a expressão no lado esquerdo é um L-value no momento da derivação. Por exemplo, as seguintes expressões são válidas em C--:

```
a[i]; //a[i] não é um L value aqui
a[i] - x; // a[i] é um L-value aqui
X- 8 + (y - 6); // x e y são L-value e (y - 6) é uma
expressão que resulta no R-value
```

mas as seguintes não são válidas em C--

```
a[i] + b[i] = x; //a[i] + b[i] não é um L value válido
x - 8 + y = 6; // (8 + y) não é um L-value. + tem
precedência sobre =
```

- A manipulação correta desses casos requer mais do que um token de look-ahead ou um ou mais tokens do tipo look-behind. Por agora, a solução é desprezar o problema e traduzir a gramática LL(1) ou LR(1) **quase** equivalente. Ou seja, a gramática será equivalente à linguagem C, com exceção de que aceitará strings do tipo $a[i] + b[i] = \text{expr}$. Portanto, a árvore de sintaxe abstrata aceita a atribuição de uma expressão a outra expressão. No próximo trabalho, ao realizar a análise semântica e a geração de código, o compilador deverá verificar se a expressão do lado esquerdo de uma atribuição é um L-value.
- A conversão de gramática C- para LL(1) ou LR(1) será **a tarefa mais importante e complexa** dessa parte do trabalho. Mesmo verificando os conjuntos FIRST e FOLLOW é recomendável testar a sua gramática manualmente para verificar se suas produções derivam corretamente de alguns programas C-. Por exemplo:

```
int a;
float b;
int calcula(int k, int h){
    println(k, "foram calculados");
}
int main(){
    x - y + u * 5;
    calcula(k, 7);
    return 0;
}
asdsa
```

- Uma vez verificado que a sua gramática é LL(1) ou LR(1) e deriva **a mesma linguagem da GLC C--**, então implemente o parser. Se sua escolha é por um **parser recursivo descendente**, gere-o a partir de uma gramática LL(1). Se um gerador de parsers for utilizado, tanto a gramática LL(1) ou LR(1) podem ser fornecidas.

Especificações para o gerenciador de erros sintáticos:

- É preciso garantir que o parser pode derivar todas as construções da linguagem C--, e suas combinações, antes de adicionar o código de recuperação de erros
- Após ter certeza de que o parser funciona, você deve implementar uma recuperação de erros com um esquema de *phrase-level* simplificado (para um único token faltante ou incorreto). Não é necessário, mas pode valer pontos extras, apresentar esquemas mais complexos de recuperação de erros. Encontre casos simples de recuperação a partir de um único token esperado, mas não encontrado. No mínimo o conjunto FOLLOW para sincronizar o parser. Quando o parser encontrar um erro, ele deve imprimir uma mensagem (em stderr), tentar se recuperar e continuar o parsing. Por exemplo, após detectar o seguinte erro:

ID LPARENT ID SEMICOLON

o parser deve imprimir uma mensagem como “programa.34:8 erro sintático. Falta fechar parênteses na chamada do procedimento” e continuar o parsing como se o parênteses estivesse no local esperado. Emita mensagens com significado para **stderr**, contendo o número da linha e coluna em que aconteceu o erro. Erros léxicos devem gerar mensagens de erro, mas não devem terminar o parsing. Busque o próximo token válido e o retorne.

Especificações para o gerador de código intermediário

- Somente adicione o código para a construção da árvore de sintaxe abstrata após
 - i) Implementar todo o parser, no caso de construir um parser recursivo descendente; ii) ou gerar e entender como funciona a interface do gerador de parsers para inserção de ações.
- A função para o primeiro não-terminal da gramática (*Program*) **deverá retornar um ponteiro para o nó raiz da árvore de sintaxe abstrata PROGRAM** indicado na tabela a seguir. Essa é a única interface do analisador sintático com o compilador
- Implemente as estruturas e a construção da árvore de sintaxe abstrata de forma independente do parser. Esse apenas deve conter o código necessário para montar a árvore. **ATENÇÃO: apenas construa a árvore NÃO verifique nada da linguagem. Isso é tarefa para o analisador semântico!!!!**
- Insira ações no parser para criar uma árvore com uma estrutura equivalente à dos nós a seguir:

	Nó	Formas possíveis
1	PROGRAM	[FUNCTION_LIST, TYPE_LIST, VAR_LIST]
2	VAR_LIST	-[NAME_DECL, VAR_LIST] -NULL
3	NAME_DECL	[TYPE, ID]
4	FUNCTION_LIST	-[TYPE, ID, VAR_LIST, VAR_LIST, STATEMENT_LIST, FUNCTION_LIST] -NULL
5	TYPE_LIST	-[VAR_LIST, ID, TYPE_LIST] -NULL
6	TYPE	-[ID, SIZE] -[PRIMITIVE, SIZE]
7	STATEMENT_LIST	-[STATEMENT, STATEMENT_LIST] -NULL
8	STATEMENT	-IF; -WHILE -BREAK -PRINTLN -READ -RETURN -STATEMENT_LIST -CALL -EXP
9	IF	-[EXP, STATEMENT, STATEMENT]
10	WHILE	-[EXP, STATEMENT]
11	BREAK	-NULL
12	PRINTLN	-[EXP_LIST]
13	READ	-EXP
14	RETURN	-EXP
15	EXP_LIST	-[EXP, EXP_LIST] -NULL
16	EXP	-ID -NUMBER -LITERAL -CHAR -NAME_EXP -ADDRESSVALUE -POINTER_ALUE -ARRAY -ASSIGN -RELATIONAL_OP -ADDITION_OP -MULTIPLICATION_OP -BOOLEAN_OP -NOT SIGN TRUE FALSE

17	ASSIGN	-[EXP, EXP]
18	NAME_EXP	-[EXP, ID]
19	ADDRESS_VALUE	-[EXP]
20	POINTER_ALUE	-[EXP]
21	ARRAY	-[EXP, EXP]
22	CALL	-[ID, EXP_LIST]
23	RELATIONAL_OP	-[OP_R, EXP, EXP]
24	ADDITION_OP	-[OP_ADD, EXP, EXP]
25	MULTIPLICATION_OP	-[OP_MUL, EXP, EXP]
26	BOOLEAN_OP	-[OP_BOOL, EXP, EXP]
27	TRUE	-NULL
28	FALSE	-NULL
29	NOT	-[EXP]

- Procure adicionar ao parser um código claro e simples para a construção e encadeamento dos nós acima. Escolha qualquer estrutura de dados para implementar a árvore. Mas lembre-se que a análise deverá percorrê-la eficientemente, encontrando padrões sintáticos. Esse módulo poderá ser desenvolvido em C++ de forma orientada a objetos. Os construtores a seguir exemplificar uma implementação em Java:

```

Program(FunctionList fl, VarList vll, TypeList tl);
VarList(NameDecl nd, VarList vl);
NameDecl(Type t, Identifier id);
functionList(Type t, Identifier id, VarList vl, VarList
localVar, StmtList sl, FunctionList next);
StmtList(Statement s, StmtList next);
ExpList(Exp e, ExpList next);
...

abstract class Statement;

if(Exp e, Statement s1, Statement s2);
while(Exp e, Statement sl);
...

abstract class Exp;

assign(Exp v, Exp e);
Call(Identifier id, ExpList e);
Identifier(String s);
Name(Exp base, Identifier id);
Number(Float n);

```

```

Number(Int n);
Literal(String literal);
Addition_OP(int op, Exp e1, Exp e2);
Relational_OP(int op, Exp e1, Exp e2);
Not(Exp e);

```

Entrada e saída do compilador:

- Você deve imprimir todos os não-terminais de sua gramática LL(1) que estiver sendo processado e **TODO** terminal que for casado com o token de entrada. Por exemplo, dado o seguinte programa:

```

int main(){
    float i;
    x = 5 -h;
}

```

O parser deve mostrar como saída como o seguinte:

```

Program
FunctionDecl
Type
Match: Int
Match: ID.main
Match: LEFTPAR
FormalList
Match: RIGHTPAR
Match: LEFTBRA
VarDecl
Type
Match: FLOAT
IdList
Match: ID.i
Match: SEMICOLON
VarDecl
Stmt
Expr
Expr

```

```

Primary
Match: Id.x
Match: ASSIGN
Expr
Expr
Primary
Match: NUMINT.5
BinOP
Match: MINUS
Expr
Primary
Match: ID.h
Match: ID.SEMICOLON
Match: ID.RIGHTBRA
Match: ID.EOF

```

- Crie uma interface com estrutura de árvore para gerar uma impressão hierárquica dos nós da árvore. Por exemplo, a sentença; `if(a < b) m() else c = k + 1` deve retornar:

```

-IF
  -LESSTHAN
    -ID.a
    -ID.b
  -CALL
    -ID.m
  -ASSIGN
    -NAME
      -ID.C
    -ADD_OP
      -ID.K
      -NUMBER.1

```

Funcionalidade opcional:

Não adicione ela até que as funcionalidades obrigatórias estejam concluídas e corretas: Laços do tipo **FOR**

Gramática C–

Vide descrição da parte 1